CS-860: ARTIFICIAL INTELLIGENCE

- **Planning and Search**
  - **Conceptual Model**
  - **Search Problems**

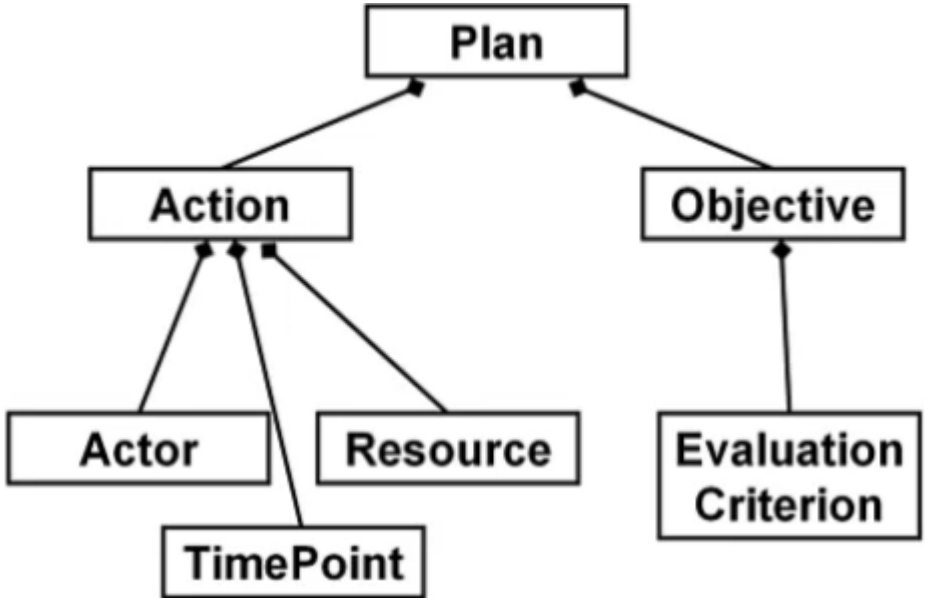
Text : Artificial Intelligence a Modern Approach
by Stuart Russel and Peter Norvig

INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

+92 332 908 9929

Conceptual Model

- Theoretical device for describing the elements of a problem
- Advantages
  - Explaining basic concepts
  - Clarifying assumptions
  - Analyzing requirements
  - Proving semantic properties
- Disadvantages
  - Efficient algorithms and computational concerns



kfaraz@gmail.com

State Transition System

- A state-transition system is a 4-tuple $\Sigma = (S,A,E,\gamma)$, where:
  - S= $\{s_1,s_2,,... \}$ is a finite or recursively enumerable set of states;
  - A = $\{a_1,a_2,... \}$ is a finite or recursively enumerable set of actions;
  - E = $\{e_1,e_2,... \}$ is a finite or recursively enumerable set of events;

    and
  - $\gamma : S \times (A \cup E) \rightarrow 2^s$ is a state transition function.

    and
- if $a \in A$ and $\gamma(s, a) \neq \varnothing$ then a is applicable in s
- applying a in s will take the system to $s' \in \gamma(s, a)$

State Transition System

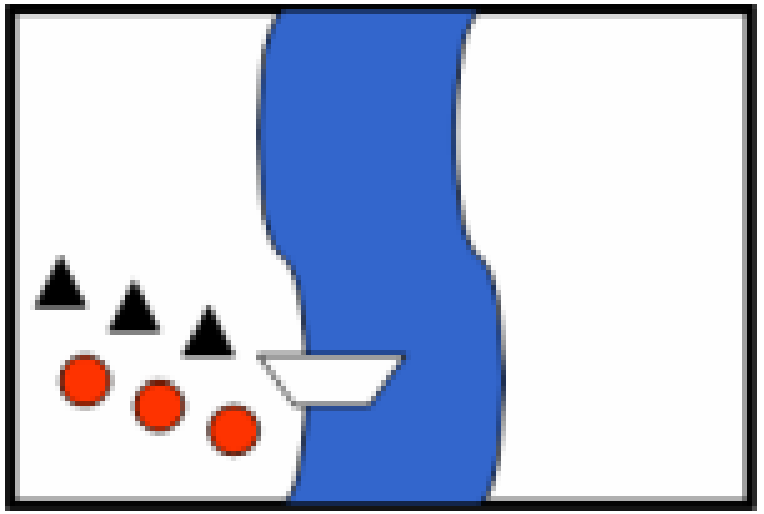
- A state-transition system $\Sigma = (S,A,E,\gamma)$ can be represented by a directed labelled graph $G = (N_G, E_G)$

  where:

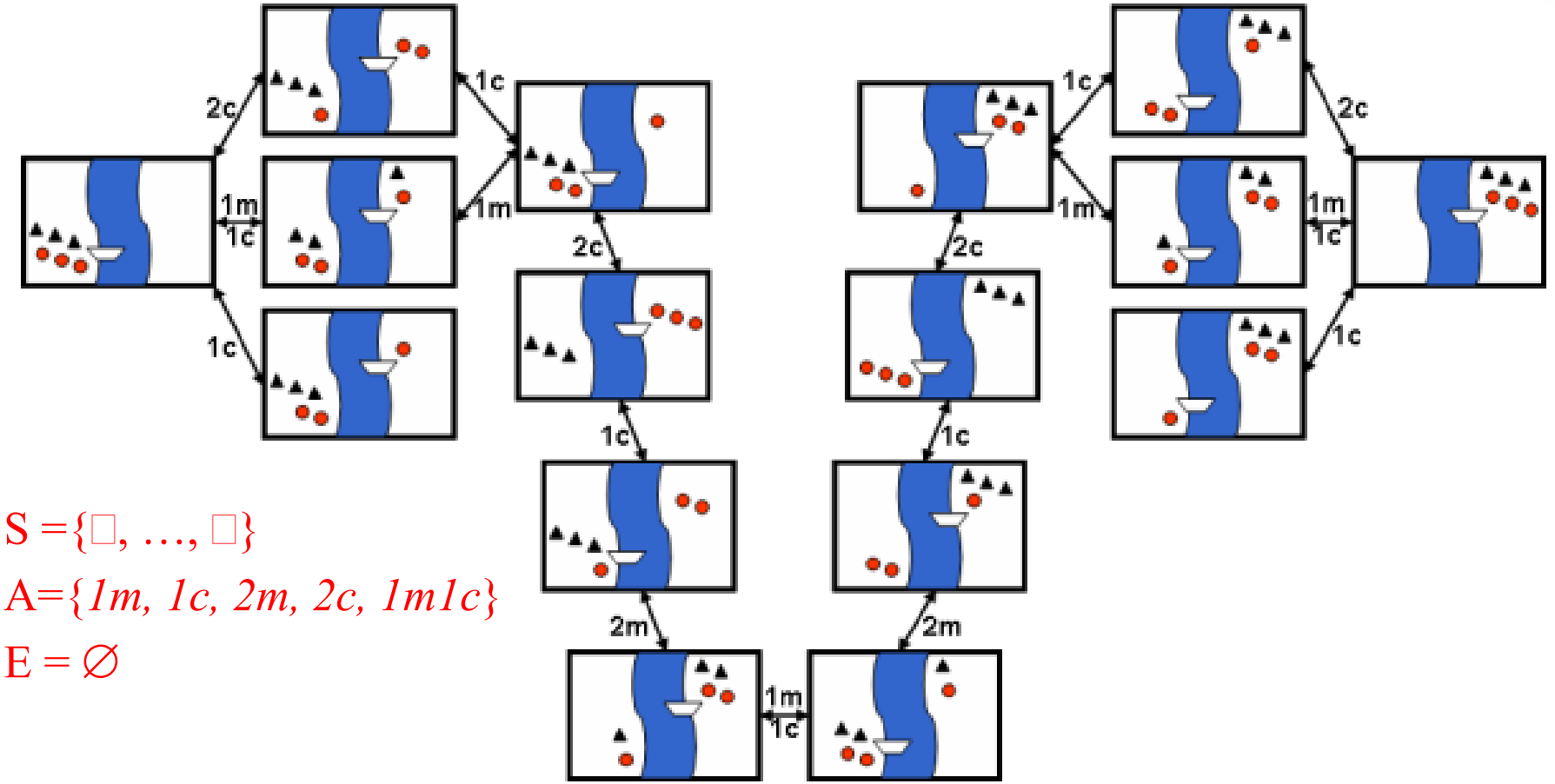
  - the nodes correspond to the states in S, i.e., $N_G = S$; and
  - there is an arc from $s \in N_G$ to $s' \in N_G$, i.e., $s \rightarrow s' \in E_G$, with label $u \in (A \cup E)$ if and only if $s' \in \gamma(s, u)$

State Transition System - Example

- In the missionaries and cannibals problem, three missionaries (black triangles) and three cannibals (red circles) must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.



State Transition System - Example



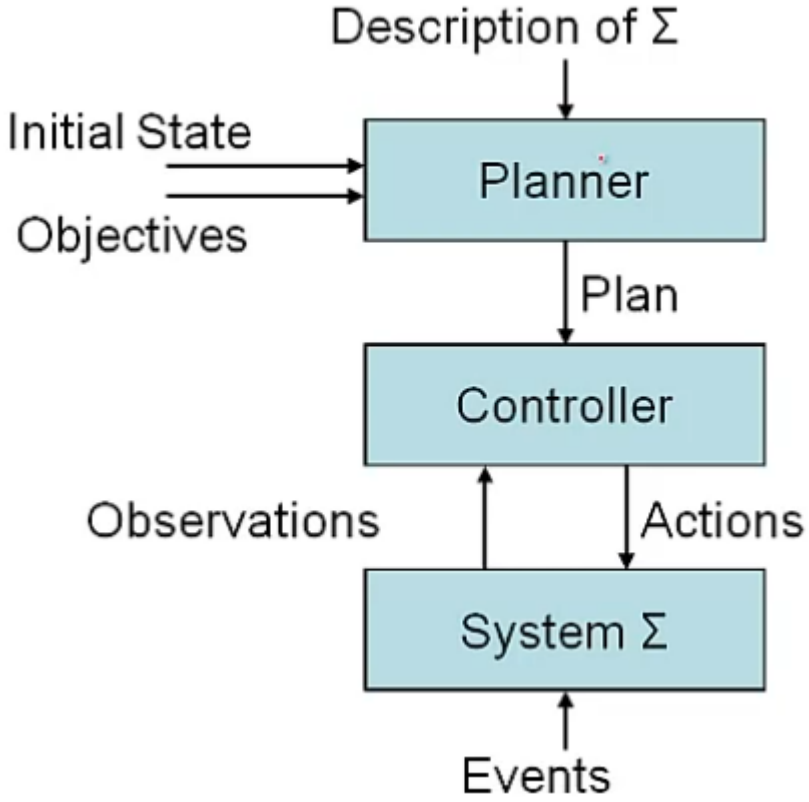
- S ={□, …, □}
- A={*1m, 1c, 2m, 2c, 1m1c*}
- E = ∅

Objectives and Plans

- State Transition System: describes all ways that a system can evolve
- Plan: a structure that gives appropriate actions to apply to achieve some objective when starting from a given state
- Types of Objectives:
  - goal state: s_g or a set of goal states S_g
  - satisfy some conditions over the sequence of states
  - optimize utility function attached to states
  - task to be performed

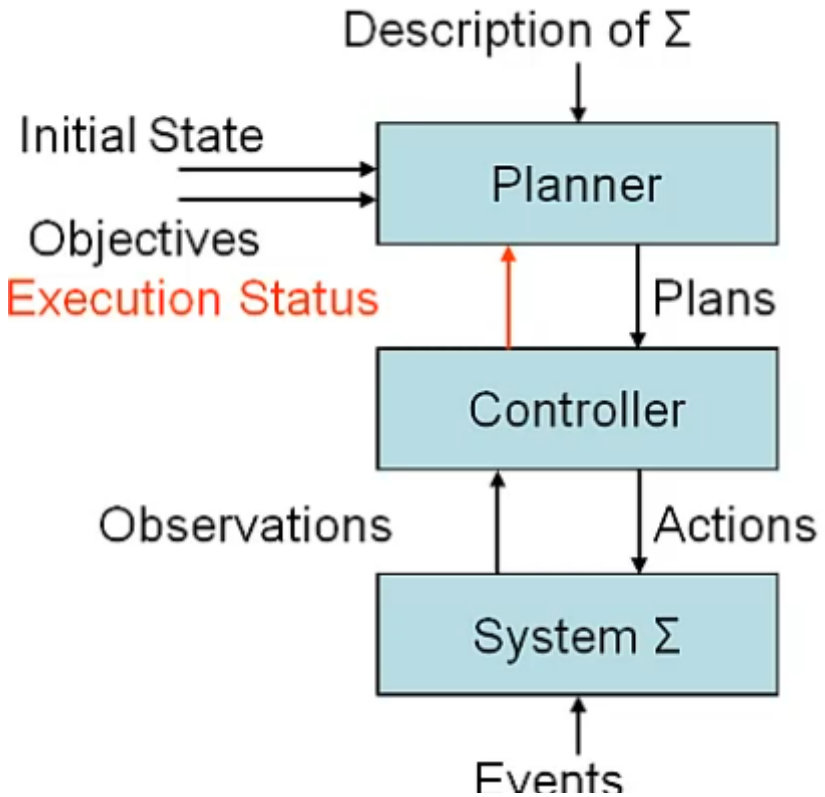
State Transition System – Objectives and Plans

- planner:
  - given: description of Σ, initial state, objective
  - generate: plan that achieves objective
- controller:
  - given: plan, current state (observation function) $\eta : S \rightarrow O$)
  - generate: action
- state-transition system:
  - evolves as actions are executed and events occur



State Transition System - Objectives and Plans

- problem: real world differs from model described by Σ
- more realistic model: interleaved planning and execution
  - plan supervision
  - plan revision
  - re-planning
- dynamic planning: closed loop between controller and planner
  - execution status



Type of Problems

Toy Problems/Puzzles

- concise and exact description
- used for illustration purposes (e.g., mostly in this course)
- used for performance comparisons

Real-World Problems

- no single agreed upon description
- people care about the solutions

Planning and Search

Search Problems

- initial state
- set of possible actions/applicability conditions
  - successor function: state → set of <action state>
  - successor function + initial state = state space
  - path (solution)
- goal
  - goal state or goal test function
- path cost function
  - for optimality
  - assumption: path cost = sum of step costs

kfaraz@gmail.com

Planning and Search

- Conceptual Model
- **Search Problems**
  - Types
  - Components
  - **Successor Function**

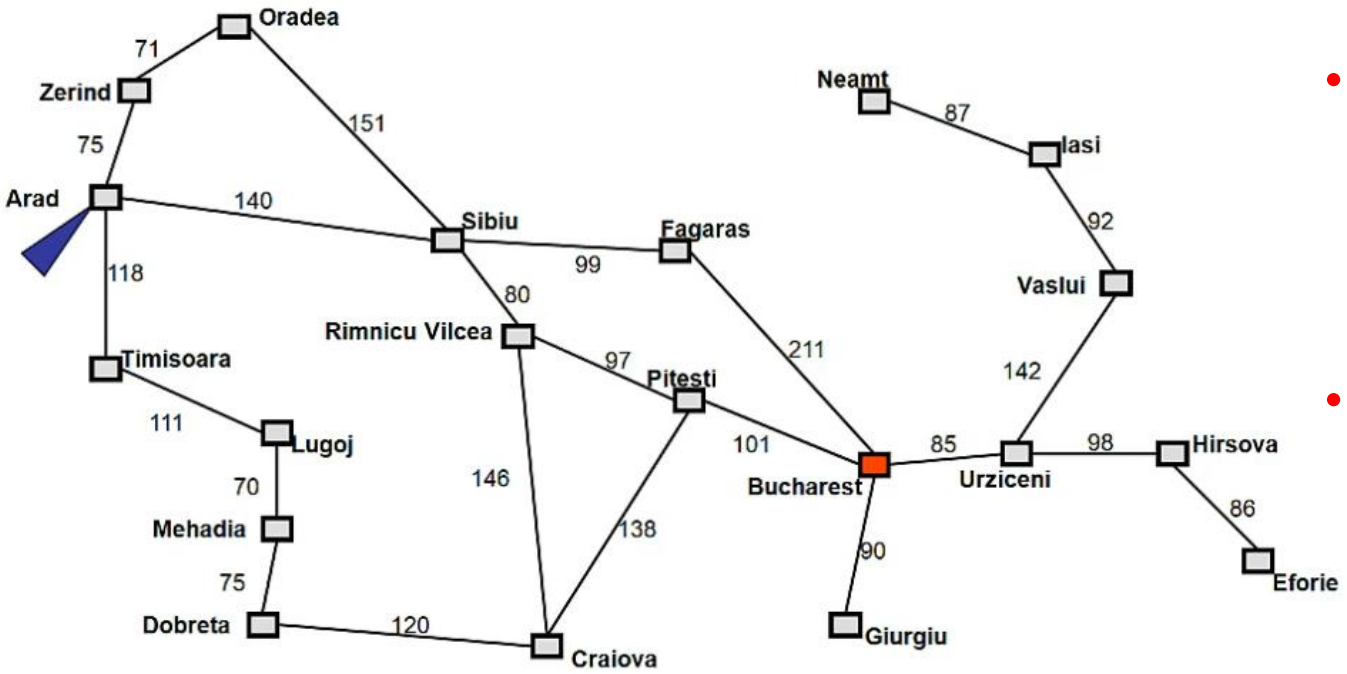
Search Problems – Successor Function

state	set of <*action*, *state*>
(L:3m,3c,b-R:0m,0c) →	{<2c, (L:3m,1c-R:0m,2c,b)>, <1m1c, (L:2m,2c-R:1m,1c,b)>, <1c, (L:3m,2c-R:0m,1c,b)>}
(L:3m,1c-R:0m,2c,b) →	{<2c, (L:3m,3c,b-R:0m,0c)>, <1c, (L:3m,2c,b-R:0m,1c)>}
(L:2m,2c-R:1m,1c,b) →	{<1m1c, (L:3m,3c,b-R:0m,0c)>, <1m, (L:3m,2c,b-R:0m,1c)>}
⋮	⋮

Planning and Search

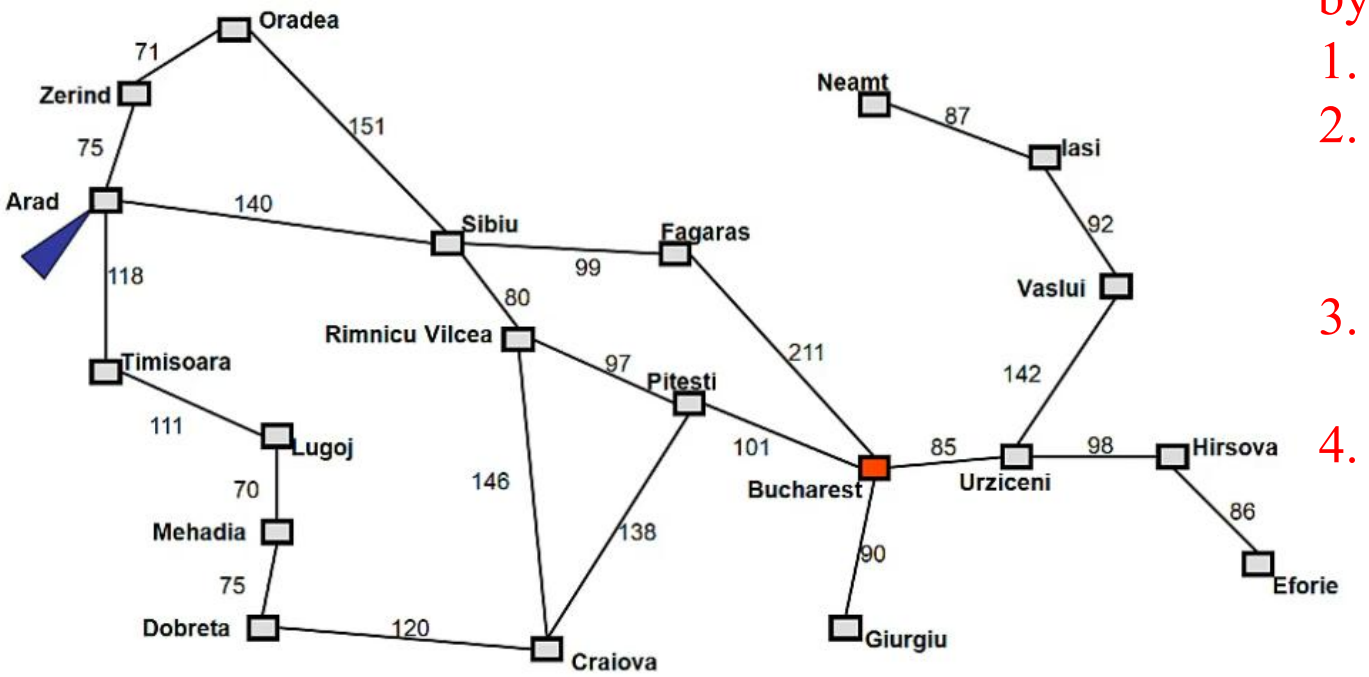
- Conceptual Model
- Search Problems
  - Types
  - Components
  - Successor Function
  - Real-World Problem

Search Problems – Real-World Problem



- Formulate goal: Be in Bucharest.
- Formulate problem: states are cities, actions: drive between pairs of cities
- Find solution: Find a sequence of cities (e.g., Arad, Sibiu, Fagaras, Bucharest) that leads from the current state to a state meeting the goal condition

Search Problems – Real-World Problem



- A search problem is defined by the
  1. Initial state (e.g., Arad)
  2. Actions (e.g., Arad → Zerind, Arad → Sibiu, etc.)
  3. Goal test (e.g., at Bucharest)
  4. Solution cost (e.g., path cost)

Search Problems – Real-World Problem

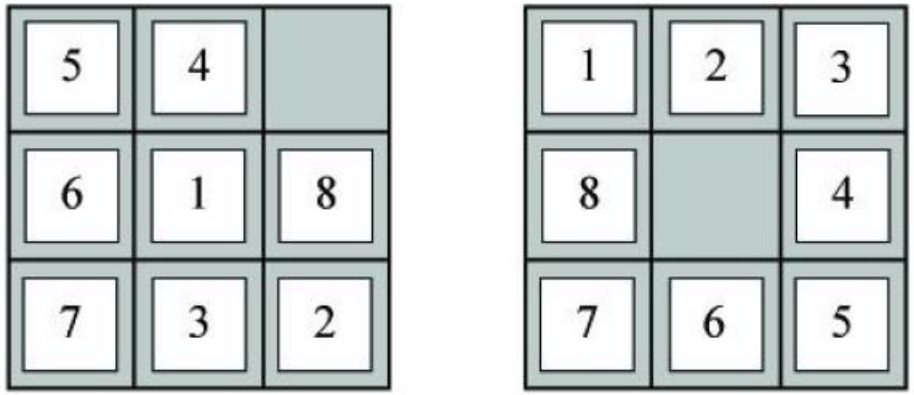
- problem formulation
  - process of deciding what actions and states to consider
  - granularity / abstraction levels
- environment assumptions
  - finite and discrete
  - fully observable
  - deterministic
  - static
- other assumptions
  - restricted goals
  - sequential plans
  - Implicit time
  - Offline planning

Search Problems – Examples

- States: number of missionaries, cannibals, and boat on near riverbank
- Initial state: all objects on near riverbank
- Actions: move boat with *x* missionaries and *y* cannibals to other side of river
  - no more cannibals than missionaries on either riverbank or in boat
  - boat holds at most m occupants
- Goal: all objects on far riverbank
- Path cost: 1 per river crossing

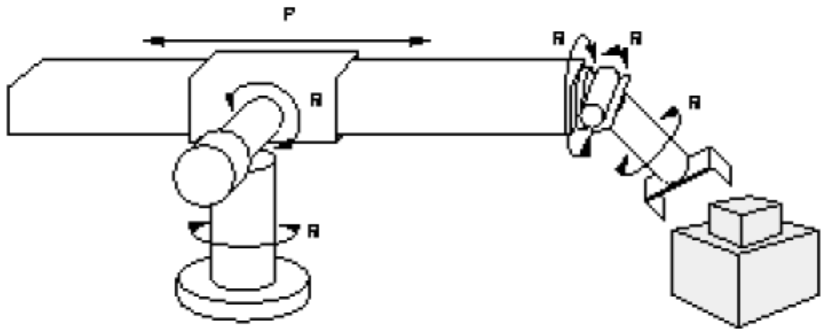
Planning and Search

Search Problems – Examples



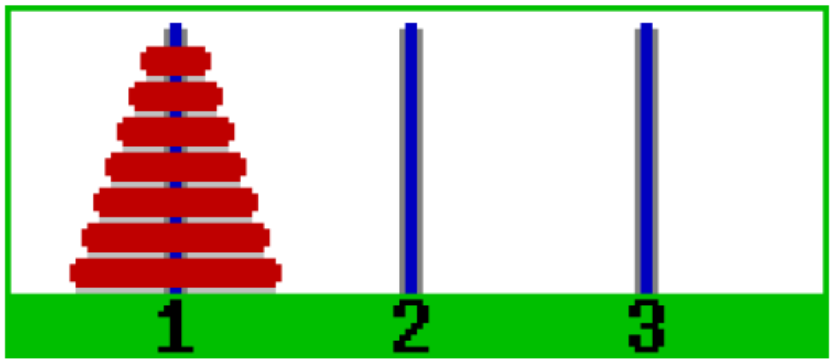
- States: tile locations
- Initial state: one specific tile configuration
- Actions: move blank tile left, right, up, or down
- Goal: tiles are numbered from one to eight around the square
- Path cost: cost of 1 per move (solution cost same as number of most or path length)

Search Problems – Examples



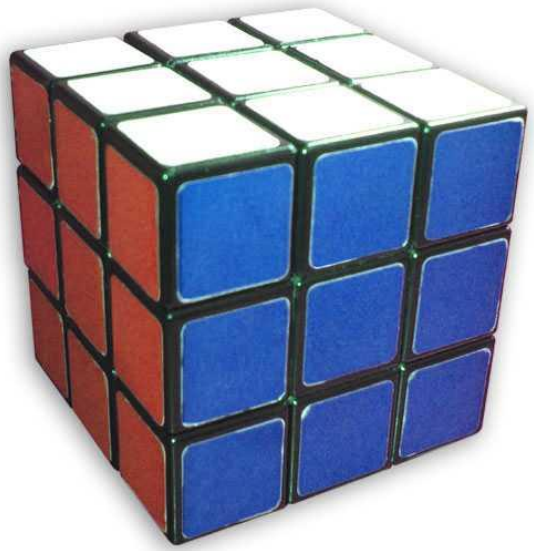
- States: real-valued coordinates of
  - robot joint angles
  - parts of the object to be assembled
- Initial state: home configuration
- Actions: rotation of joint angles
- Goal: complete assembly
- Path cost: time to complete assembly

Search Problems – Examples



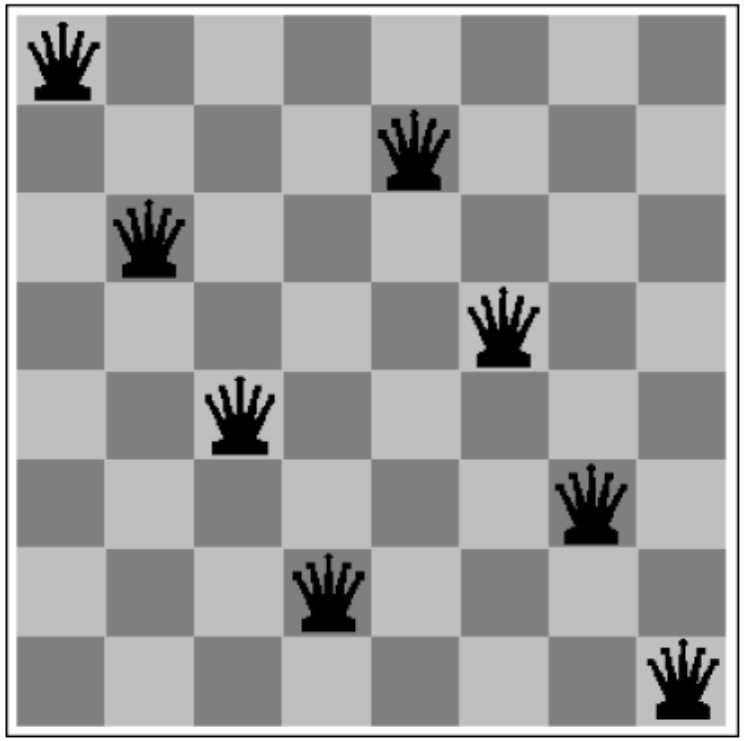
- States: combinations of poles and disks
- Initial state: start configuration
- Actions: move disk x from pole 1 to pole 3 subject to constraints
  - cannot place disk on top of smaller disk
  - cannot move disk if other disks on top
  - only one disk can be moved at a time
- Goal: disks from largest (at bottom) to smallest on goal pole
- Path cost: 1 per mov

Search Problems – Examples



- States: list of colors for each cell on each face
- Initial state: one specific cube configuration
- Actions: rotate row *x* or column *y* or face *z* direction
- Goal: configuration has only one color on each face
- Path cost: 1 per move

Search Problems – Examples



- States: locations of 8 queens on chess board
- Initial state: one specific queens configuration
- Actions: move queen *q* to row *x* and column *y*
- Goal: no queen can attack another (cannot be in same row, column, or diagonal)
- Path cost: 1 per move

Search Problems – Nodes

- search nodes: the nodes in the search tree
- data structure:
  - *state*: a state in the state space
  - *parent node*: the immediate predecessor in the search tree
  - *action*: the action that, performed in the parent node's state, leads to this node's state
  - *path cost*: the total cost of the path leading to this node
  - *depth*: the depth of this node in the search tree

Search Problems – Algorithm

```
function treeSearch (problem, strategy)
    fringe ← {new searchNode (problem.initialState)}
    loop
        if empty(fringe) then return failure
        node ← selectFrom (fringe, strategy)
        if problem.goalTest (node.state) then
            return pathTo (node)
        fringe ← fringe + expand (problem, node)
```

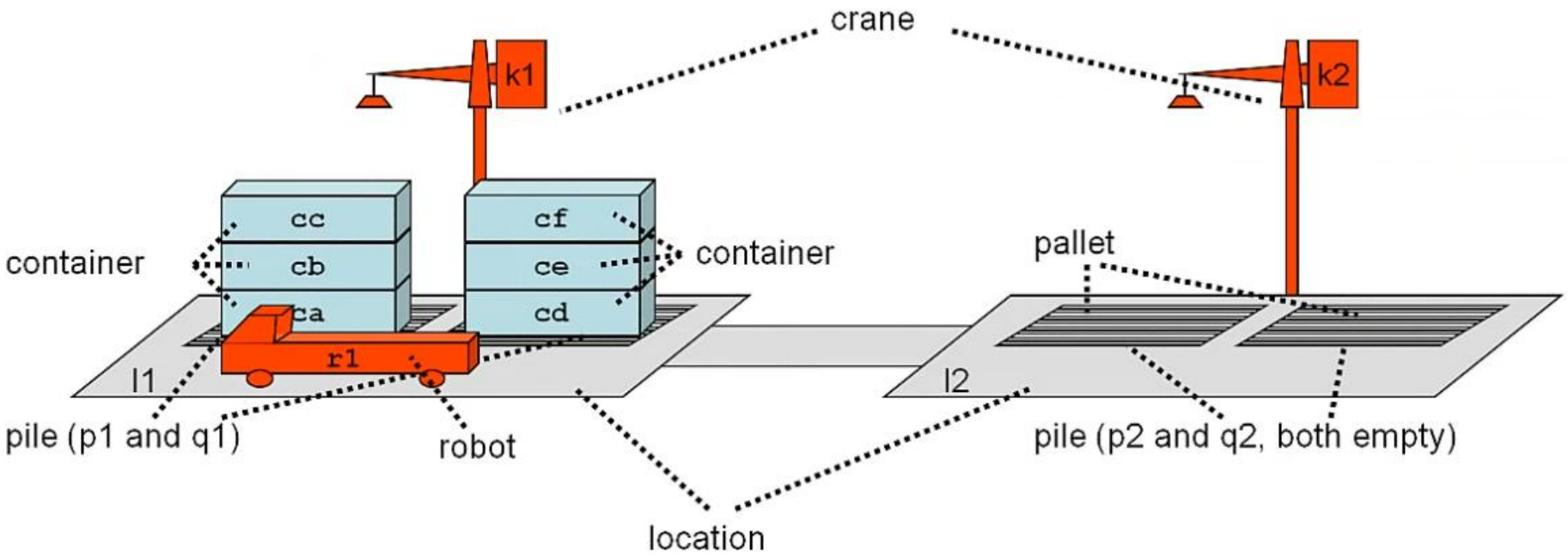
Search Problems – Real-World Problem

- search or control strategy: an effective method for scheduling the application of the successor function to expand nodes
  - selects the next node to be expanded from the fringe
  - determines the order in which nodes are expanded
  - aim: produce a goal state as quickly as possible
- examples:
  - LIFO/FIFO-queue for fringe nodes
  - alphabetical ordering

Search Problems – Real-World Example

- aim: have one example to illustrate planning procedures & techniques
- informal description:
  - harbor with several locations (docks), docked ships, storage areas for containers, and parking areas for trucks and trains
  - cranes to load and unload ships etc., and robot carts to move containers around

Search Problems – Real-World Problem



Search Problems – Real-World Problem

- ***move*** robot ***r*** from location ***l*** to some adjacent and unoccupied location ***l'***
- ***take*** container ***c*** with empty crane ***k*** from the top of pile ***p***, all located at the same location ***l***
- ***put*** down container ***c*** held by crane ***k*** on top of pile ***p***, all located at location ***l***
- ***load*** container ***c*** held by crane ***k*** onto unloaded robot ***r***, all located at location ***l***
- ***unload*** container ***c*** with empty crane ***k*** from loaded robot ***r***, all located at location ***l***